SecureRS

Email: asingh@cs.up.ac.za
Website: https://github.com/AvinashSingh786/SecureRS

User Manual



DigiForS
Digital Forensic Science

About

This tool was designed for research in the field of Digital Forensics.

This prototype solution was created with Digital Forensic Readiness processes for secure storage and retrieval or potential digital evidence. This solution is generic and can be used for any application that requires secure storage. There is also an API built in that allows integration with any system or tool. From the admin panel you can create and manage API keys and routes.

Installation

This tool can be run from a docker container that can be built using the dockerfile. Alternatively, you can clone this repository and install the python requirements. This tool only works for Python3 and was tested with Python3.7. It is recommended you run this in a virtual environment to further ensure compatibility and added security.

```
$ git clone git@github.com:AvinashSingh786/SecureRS.git
$ cd SecureRS
$ python3 -m pip install --user virtualenv
$ apt-get install python3-venv python3-magic # for Linux
$ python3 -m venv venv
$ source venv/bin/activate # for Linux
$ pip3 install python-magic-bin # for Windows
$ .\venv\Scripts\activate # for Windows
(venv)$ pip install -r requirements.txt
```

Usage

Run the following commands to configure and run the engine.

```
(venv)$ python manage.py makemigrations pde # This sets up the storage engine
(venv)$ python manage.py makemigrations
(venv)$ python manage.py migrate # This creates the databases and interfaces
(venv)$ python manage.py createsuperuser # Create a super user that you will use
as the admin
(venv)$ python manage.py runsslserver 0.0.0.0:8000
```

Issues

If you get TypeError: get_available_name() got an unexpected keyword argument 'max_length' please navigate to venv/lib/ sitepackages/django/core/files/storage.base.py in the save function, line 37 remove the max_length parameter.

Features

- OTP Login + Download (TOTP, YubiKey)
- REST API for Ingestion
- Two Factor Auth
- Secure Cookies
- Integrity Verification
- Encrypted Storage
- Security Headers
- Email Config
- Session Security
- Customizable

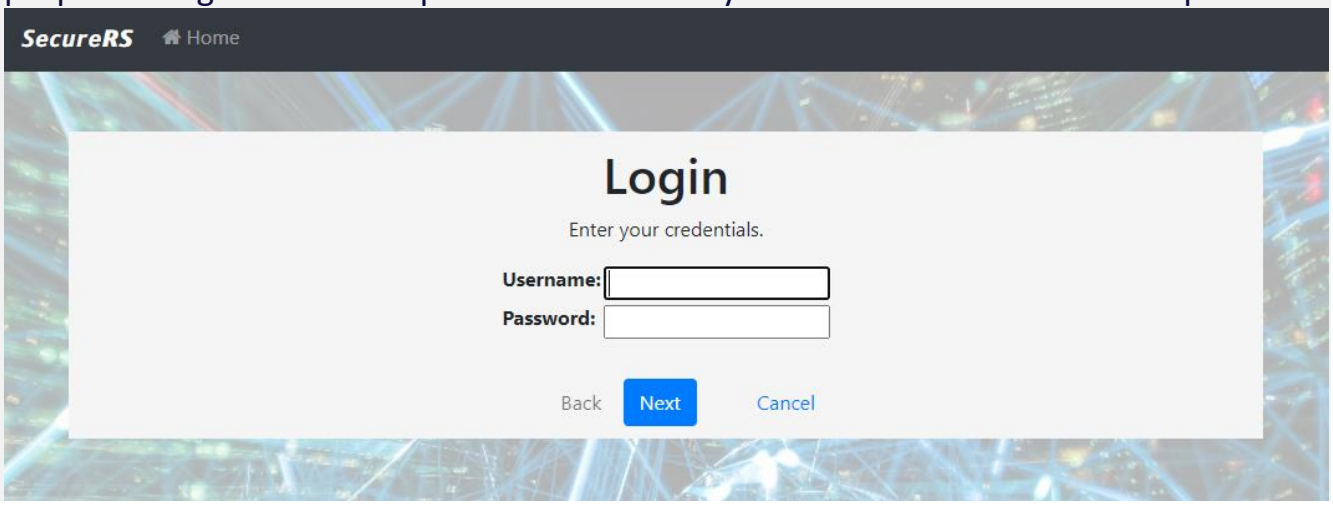
Important!

If you plan on using this tool in production, please change the following in the settings.py file:

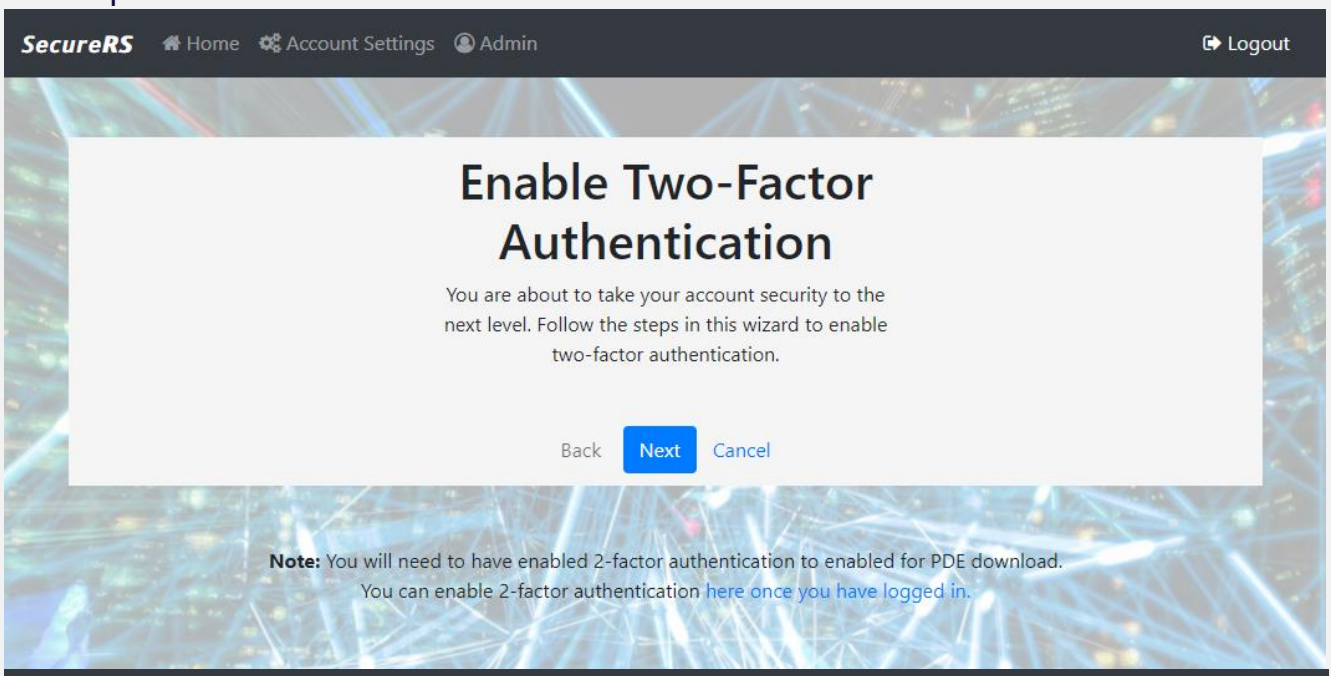
- SECRET_KEY
- DEBUG
- ALLOWED_HOSTS
- COMPANY_NAME
- DEFF_PASSWORD
- DEFF_SALT
- SESSION_SECURITY_EXPIRE_AFTER
- SESSION_SECURITY_WARN_AFTER
- EMAIL_USE_TLS
- EMAIL_HOST
- EMAIL_PORT
- EMAIL_HOST_USER
- EMAIL_HOST_PASSWORD

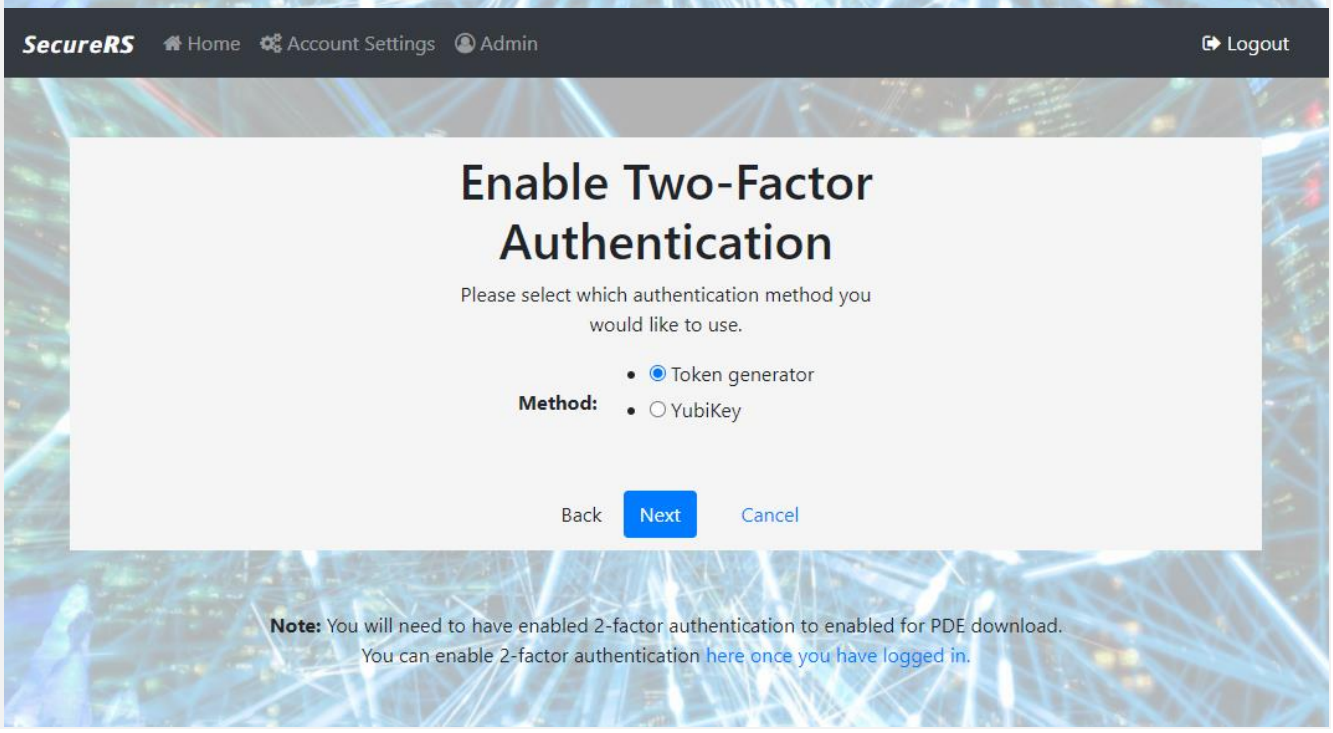
Login

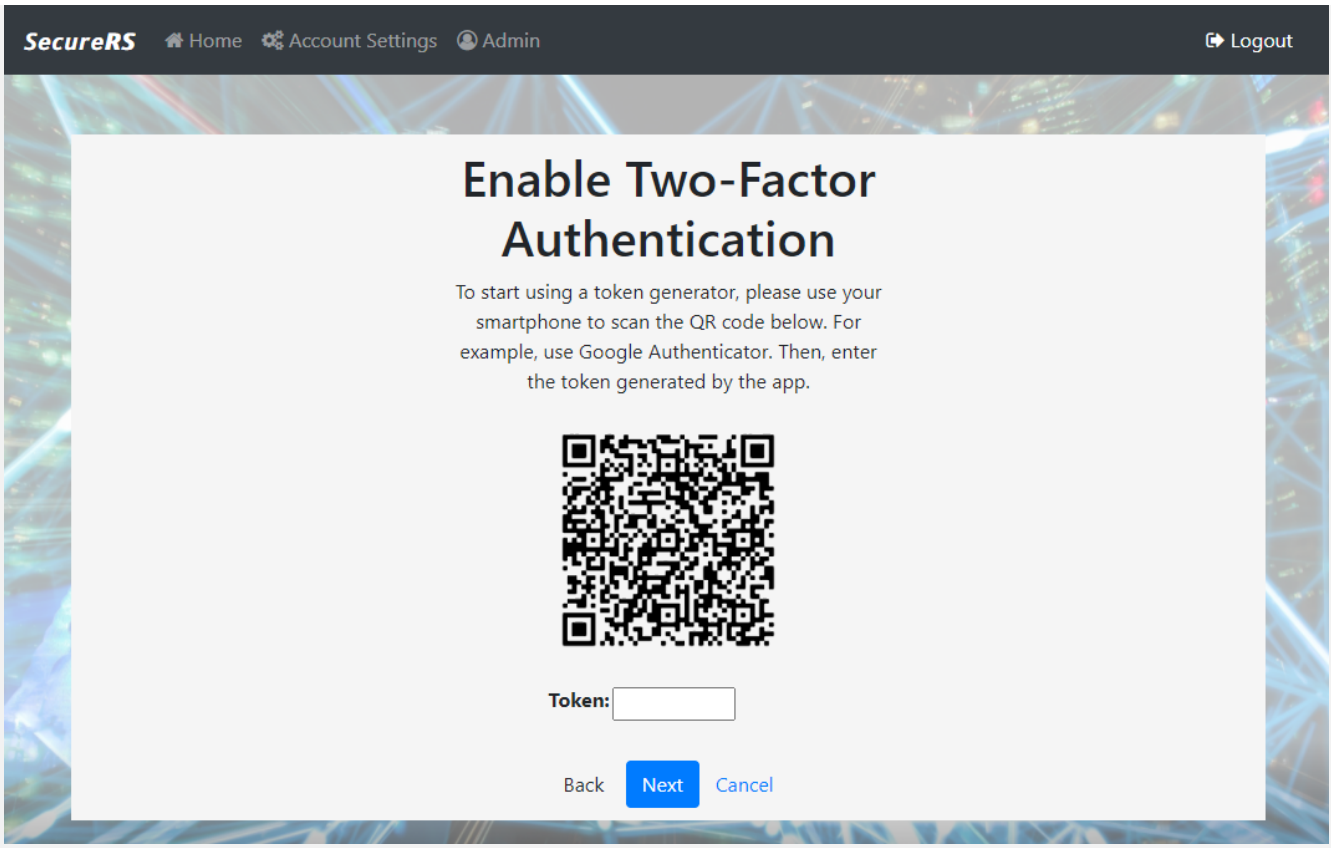
Once you have installed and setup the system you can navigate to https://localhost:8000 and will be presented with the screenshot below. If you are using a self-signed SSL certificate you will see browser errors which you can ignore for testing purposes. Login with the superuser credentials you created in the installation phase.



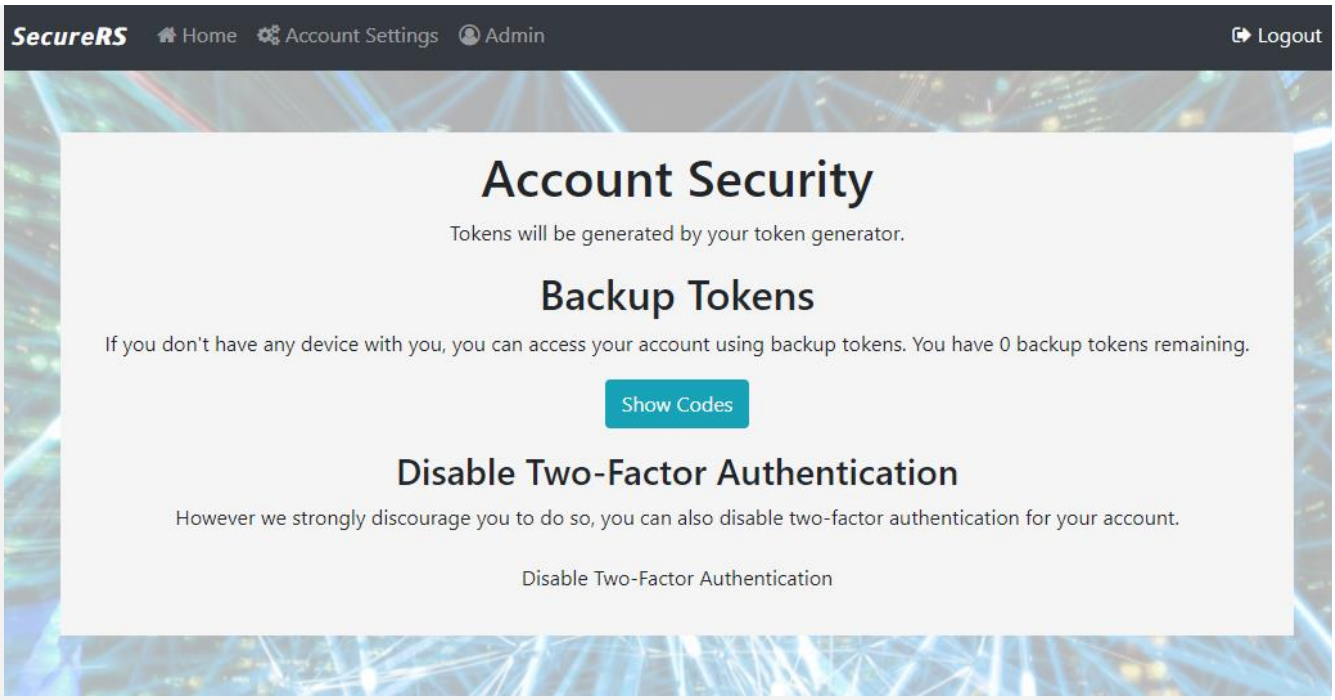
After you have logged in you will need to setup two-factor authentication by following the steps below.





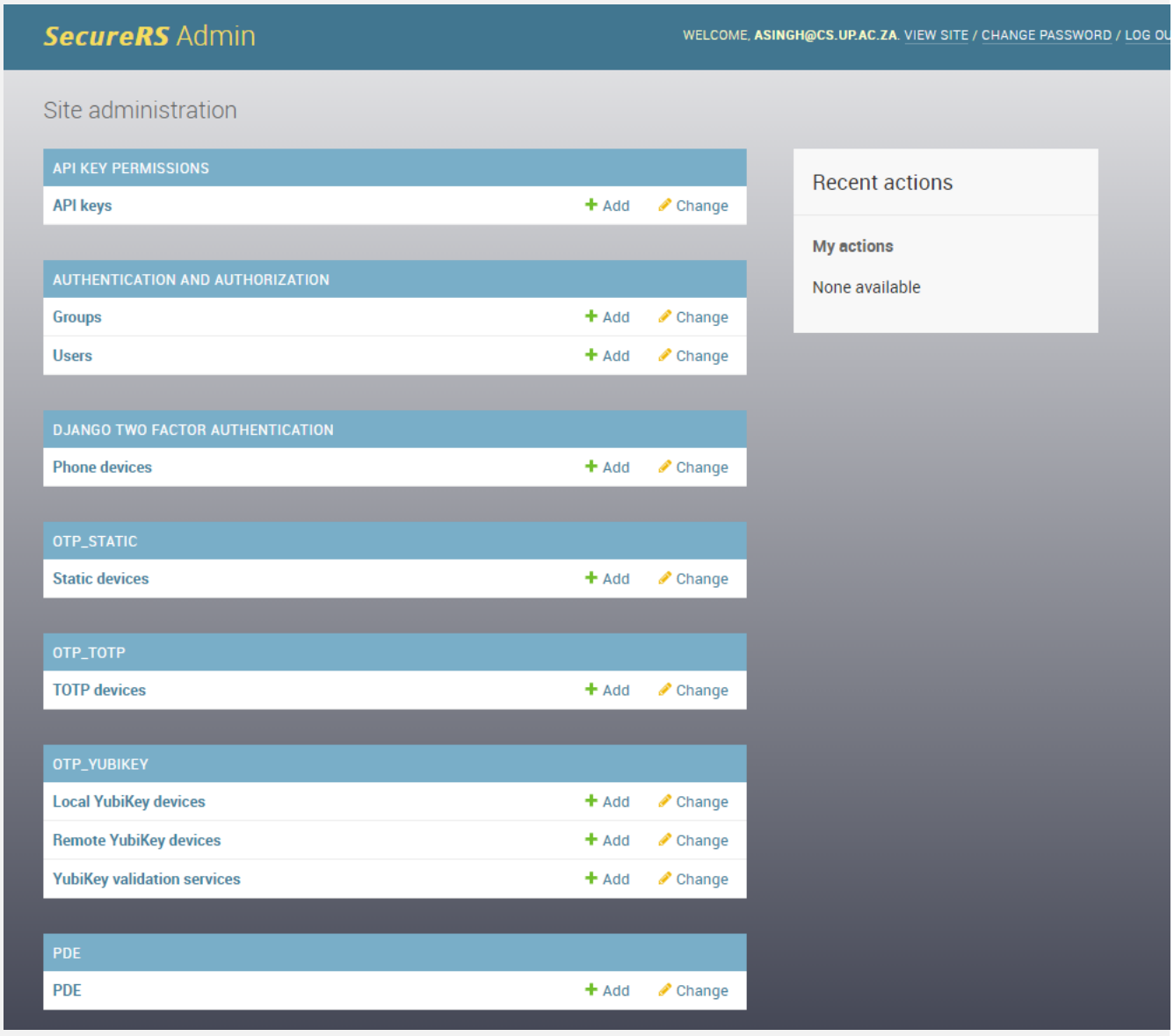


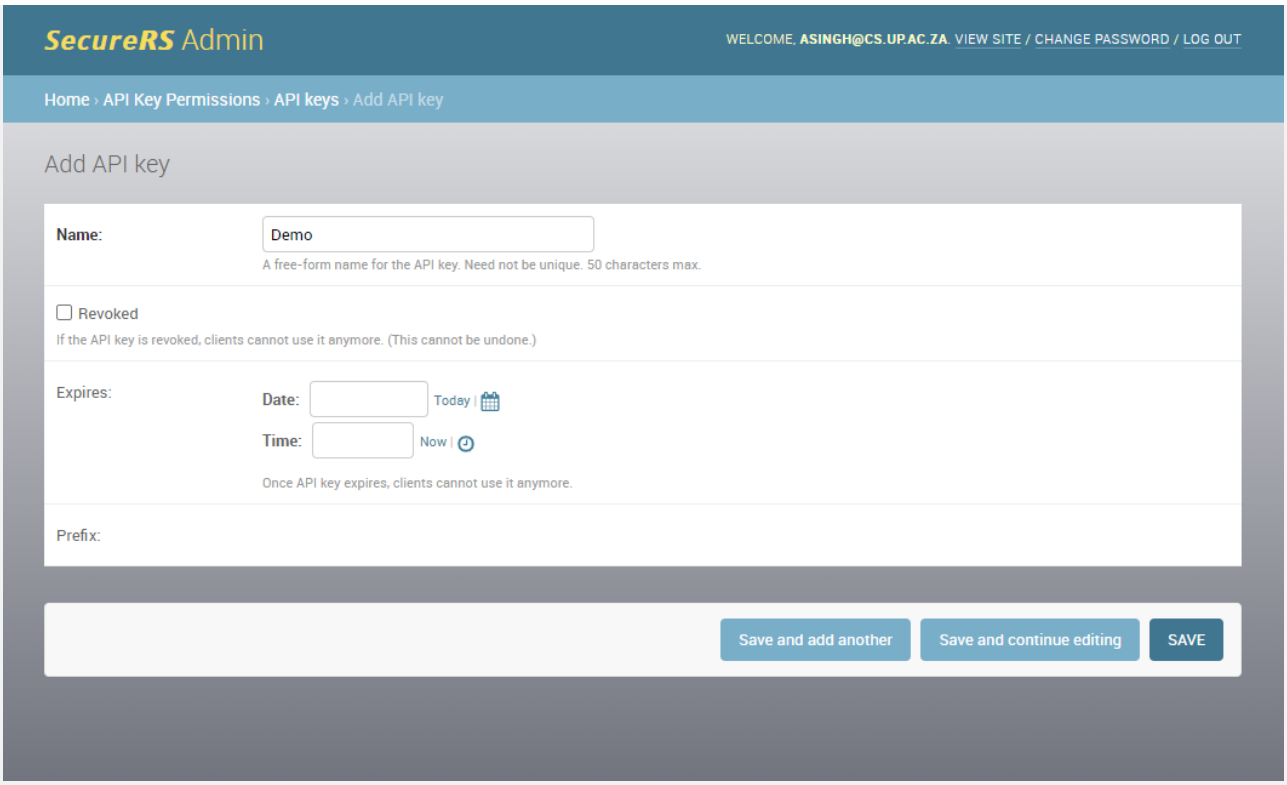
In the event you lose the 2FA app (Google Authenticator) or it resets, you should use the backup tokens. After each use of a backup token an email will be sent in order to protect your account, once a backup token has been used, it cannot be used again. You will need to enable 2FA in order to download any PDE files.


SecureRS
Home
Account Settings
Admin
Logout
Account Security
Tokens will be generated by your token generator.
Backup Tokens
If you don't have any device with you, you can access your account using backup tokens. You have 0 backup tokens remaining.
Show Codes
Disable Two-Factor Authentication
However we strongly discourage you to do so, you can also disable two-factor authentication for your account.
Disable Two-Factor Authentication

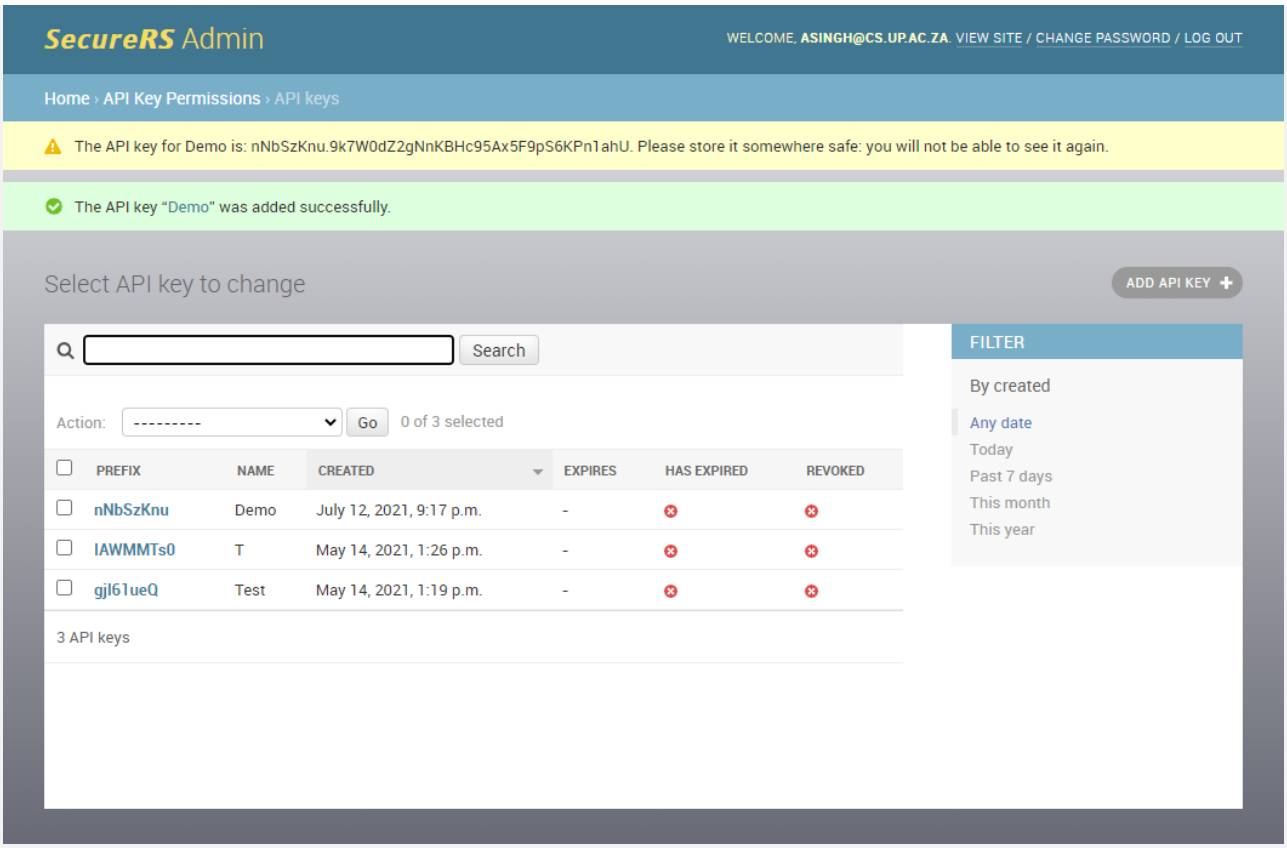
Admin

The Django Admin allows you to do a lot of powerful operations, one of which is management. Only superusers are able to view this, in order to add API keys. The next steps detail how an API key can be generated and added to the system.





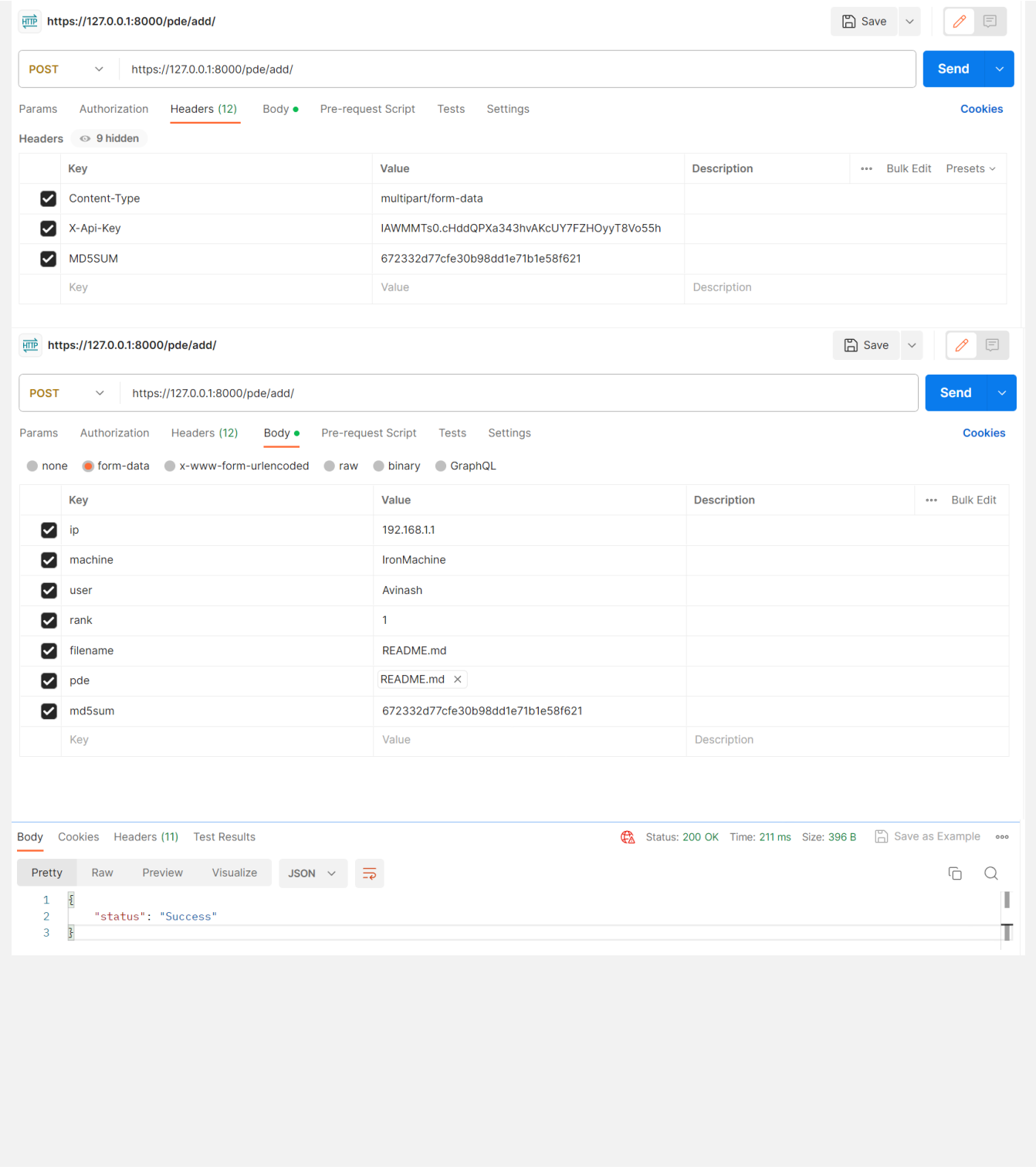
Once you have given the API key a name, you will be presented with the API Key, this will only be visible once and if the page is reloaded it will be forever lost, so ensure that you note it down. The platform also allows an API key to be revoked in the event the API key is no longer needed, but a record needs to be kept. This API key is then used for ingestion and any tool or application can used to submit data to be securely stored. This system was designed for Potential Digital Evidence (PDE) which are relatively small amounts of data, and is not meant for large disk dumps without performance penalty.



API Endpoint

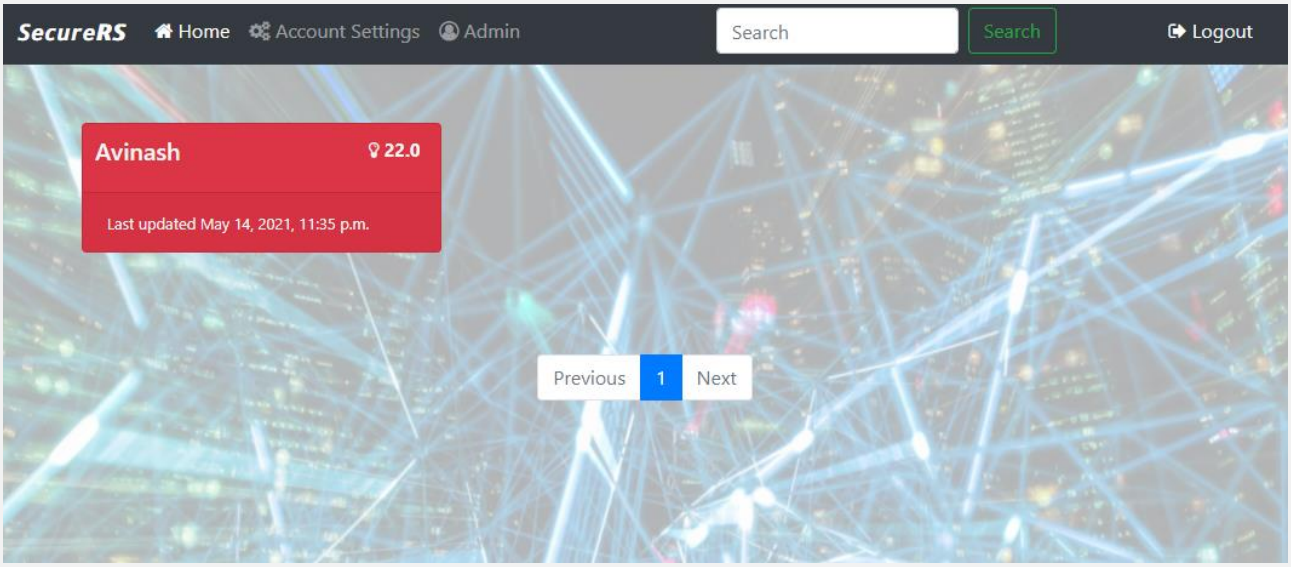
Currently, the SecureRS system only allows data ingestion as an API endpoint, this is because downloading of PDE/data needs to cater for 2-Factor authentication and verification. Therefore, this was done on the webapp instead. An API endpoint for downloading PDE in a secure manner would be in a future release. The API endpoint for data ingestion is `/pde/add/` where the HTTP headers **X-Api-Key** are used for authentication and **MD5SUM** is used for verifying the PDE file on upload. The API accepts the following POST parameters:

Name	Type	Explanation
ip	text	The IP address of the machine.
machine	text	The computer name as a unique identifier.
user	text	The user of the origin pde file.
rank	text	A number that is used to determine the suspiciousness of the pde file.
filename	text	The name of the pde file.
pde	file	The pde data itself.
md5sum	text	The md5 hash of the pde data, that is used for pde integrity verification upon ingestion.

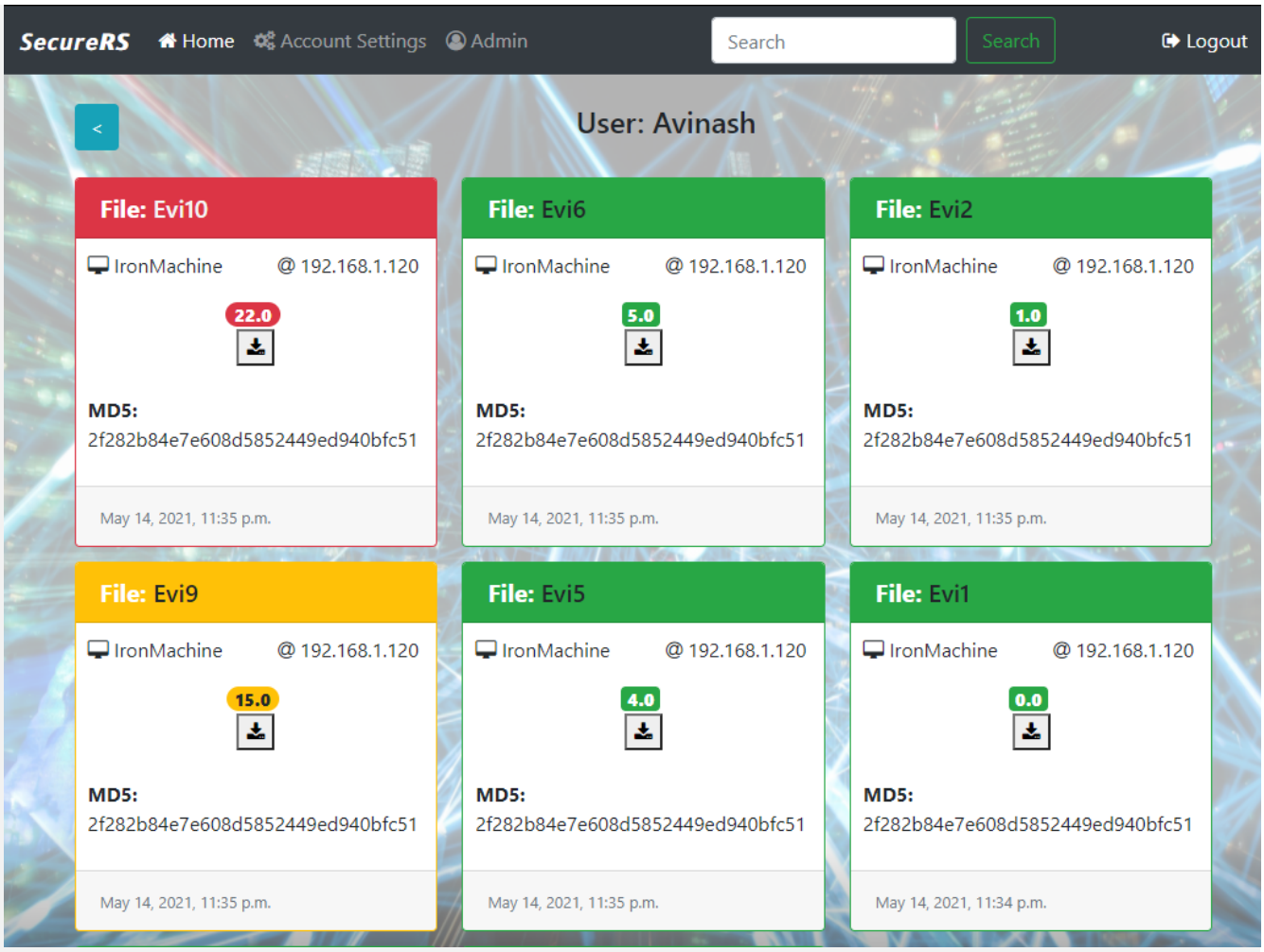

https://127.0.0.1:8000/pde/add/
Save
POST
https://127.0.0.1:8000/pde/add/
Send
Params Authorization Headers (12) Body Pre-request Script Tests Settings
Cookies
Headers 9 hidden
Key Value Description Bulk Edit Presets
Content-Type multipart/form-data
X-Api-Key IAWMMTs0.cHddQPXa343hvAKcUY7FZHOyyT8Vo55h
MD5SUM 672332d77cfe30b98dd1e71b1e58f621
Key Value Description
https://127.0.0.1:8000/pde/add/
Save
POST
https://127.0.0.1:8000/pde/add/
Send
Params Authorization Headers (12) Body Pre-request Script Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL
Key Value Description Bulk Edit
ip 192.168.1.1
machine IronMachine
user Avinash
rank 1
filename README.md
pde README.md
md5sum 672332d77cfe30b98dd1e71b1e58f621
Key Value Description
Body Cookies Headers (11) Test Results
Status: 200 OK Time: 211 ms Size: 396 B
Save as Example
Pretty Raw Preview Visualize JSON
{
"status": "Success"
}

View/UI

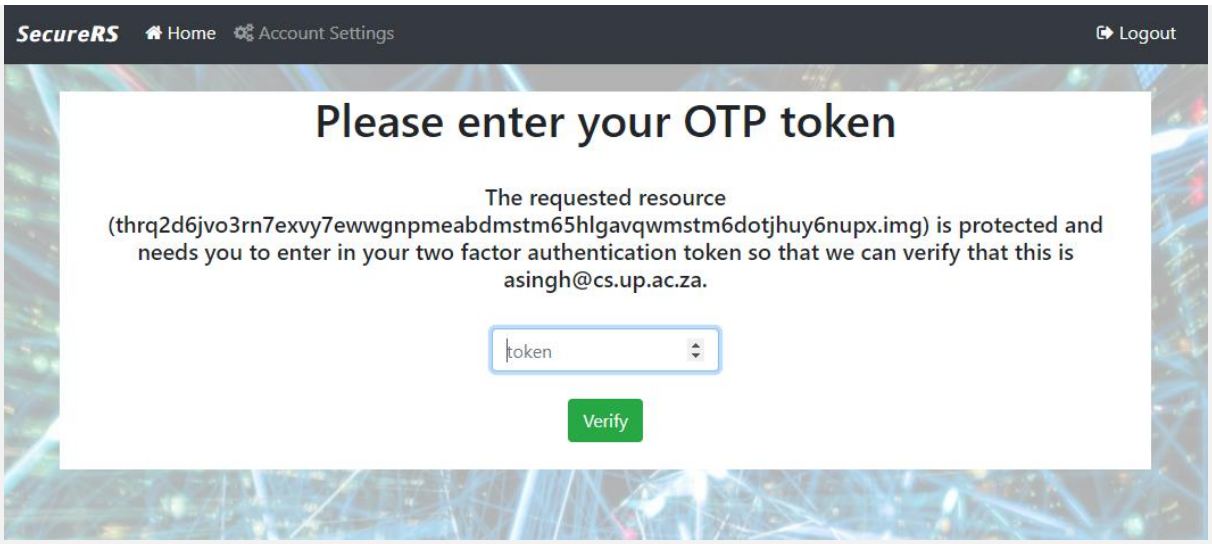
After the configuration and account setup has been completed any data submitted to the system can be seen by the superuser or investigator (through a separate standard account created in the Django Admin with limited permissions) and is separated by user for ease of access by an investigator. At a glance the latest PDE file is used to show if there are any latest issues in red which is configurable by the thresholds defined in the settings.py file.

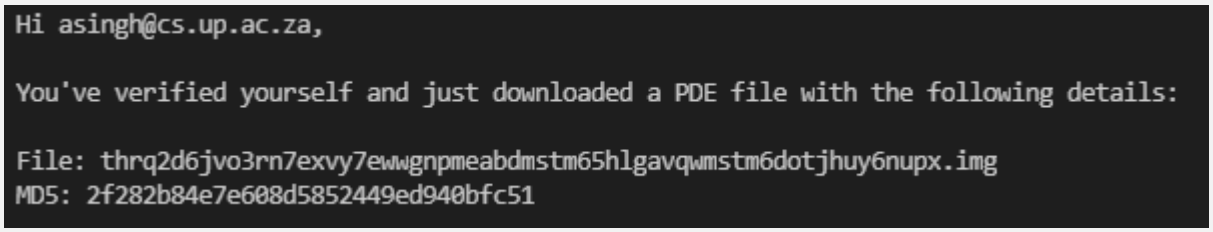


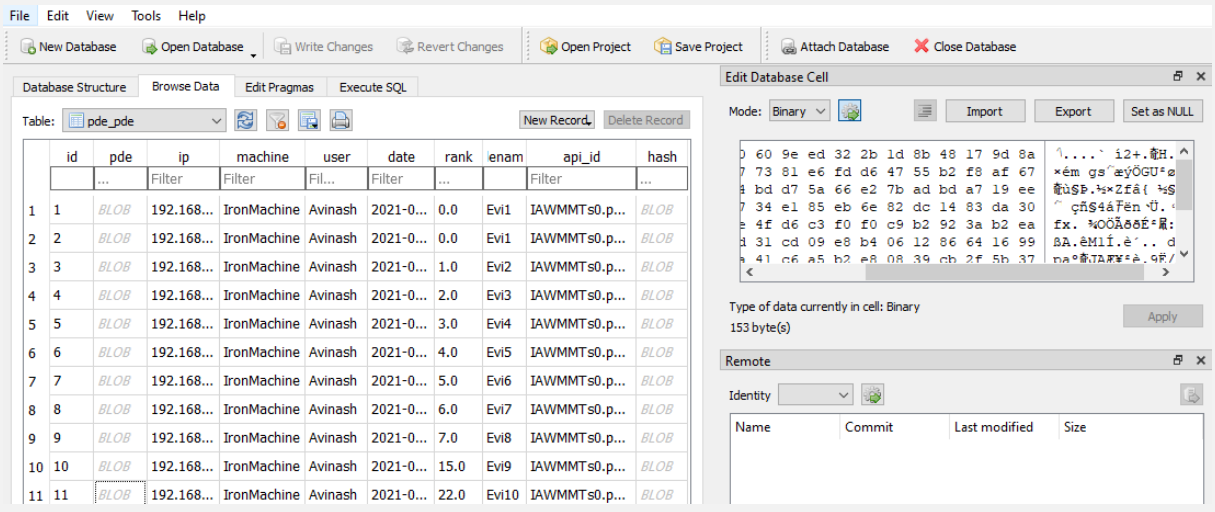
Once the investigator has selected the correct user, all the PDE files stored for that specific user is then displayed with the rank, timestamp, md5 sum, machine name, ip address, date as well as the option to download the PDE.



After the investigator has selected the PDE to download a 2FA screen will appear to validate the user as well as the privileges. Once the token is validated successfully an email will be sent with the MD5 hash, and the file will be decrypted and be downloaded to the local computer for further investigation. The last screenshot showcases the view of the database in which the Hash as well as the PDE file is stored as encrypted data in Blob format. This makes it impossible to alter the data without it being detected through the various forensic processes involved in this system.







Thank You 😊

The End